

Einführung in die Algorithmik

Übungsblatt 8

Hinweise

Das ist ein bewertetes Übungsblatt, welches für das Bestehen des Modules „Einführung in die Algorithmik“ relevant ist. Bitte beachten Sie folgende Hinweise:

- Erstellen Sie für die Abgabe des Blattes eine lesbare PDF Datei. Erstellen Sie für die Python Programme nur genau eine Datei, in welcher Sie mit Kommentaren kennzeichnen, um welche Aufgabenlösungen es sich handelt. Dies kann beispielsweise wie folgt aussehen:

```
#####
# Übung 1:
#####
# Übung 1-1
def quantumfoo():
```

- Geben Sie dieses Blatt nur in Gruppen von mindestens zwei und maximal vier Personen ab.
- Um Personen für Ihre Übungsgruppe zu finden nutzen Sie bitte die vorgesehenen Foren in StudOn oder ihr Tutorium.
- Benennen Sie die Dateien, welche Sie abgeben wollen nach folgendem Schema, wobei < und > nicht enthalten sein sollen:
 - Schema: hw<ÜbungsNr.>.<Vornamen>.<Nachname>.<Vornamen>.<Nachname>.*
 - Beispiel 3er Gruppe für Übung 2: hw2.Hongjun.Wu.Tanja.Lange.Jonathan.Katz.pdf
- Nutzen Sie zur Abgabe die dafür vorgesehene Umgebung in StudOn.
- Achten Sie auf eine saubere und lesbare Abgabe. Wir empfehlen die Verwendung eines auf T_EX basierenden Textsatzsystems wie beispielsweise L^AT_EX. Den korrigierenden Personen ist es nach eigenem Ermessen gestattet, nicht lesbare Lösungen mit 0 Punkten zu bewerten.
- Wenn Sie in Ihrer PDF-Agabe auf Ihre Programme oder Teile davon verweisen müssen, empfehlen wir die Verwendung des L^AT_EX-Pakets *minted*.
- **Bewertung:** Insgesamt können 50 Punkte pro bewertetem Übungsblatt erzielt werden. Es wird 4 bewertete Blätter geben. Sie brauchen insgesamt mindestens 100 Punkte um die Übung zu bestehen.
- Aus didaktischen Gründen sind die nutzbaren Funktionalitäten von Python eingeschränkt. Die zugelassenen Funktionalitäten finden Sie in den Einführungsfolien zu Python oder als expliziten Hinweis bei den Aufgabenstellungen.

Aufgabe:	1.1	1.2	2.3	2.4	2.5	3.6	Summe:
Punkte:	4	4	6	6	6	24	50
Ergebnis:							

Einführung in die Algorithmik

Übungsblatt 8

1 Graphentheorie

- 1.1. (4 Punkte) Auf einer Party sind n Leute und manche umarmen sich zur Begrüßung. Dann gibt es am Ende mindestens zwei Personen, welche gleich viele Umarmungen hatten. (Wir nehmen an, dass es nicht möglich ist, sich selbst zu umarmen.) Geben Sie die dazugehörige graphentheoretische Aussage an und beweisen Sie diese.
- 1.2. (4 Punkte) Jede Person auf der Party ist mit mindestens der Hälfte der Personen gut befreundet. Somit ist, direkt oder über Umwege, jeder mit jedem befreundet. (Wir nehmen an, dass mit sich selbst befreundet sein ausgeschlossen ist.) Geben Sie die dazugehörige graphentheoretische Aussage an und beweisen Sie diese.

2 Page Rank

Der in der Vorlesung vorgestellte PageRank Algorithmus kann für einzelne Knoten wie folgt berechnet werden:

$$PR_v = \sum_{u \rightarrow v} \beta \frac{PR_u}{d_G^+(u)} + (1 - \beta) \cdot \frac{1}{N}$$

Dabei ist $G = (E, V)$ ein Graph mit $u, v \in V$, $d_G^+(u)$ der Ausgangsgrad von u , $\beta \in [0, 1]$ der Dämpfungsfaktor, $N = |V|$ und PR der PageRank, welcher für jeden Knoten mit $1/N$ initialisiert wird.

Die Schreibweise mittels Matrizen ist wie folgt:

$$M' = \beta \cdot M + (1 - \beta) \left[\frac{1}{N} \right]_{N \times N}$$

Nach der Transponierung kann M'^T anschließend für die Power Iteration $PR' = M'^T \cdot PR$ verwendet werden, wobei PR in diesem Fall der Vektor aller PageRanks ist. M ist die gewichtete Adjazenzmatrix A definiert wie folgt:

$$M_{uv} = \begin{cases} \frac{1}{d_G^+(v)} & (v, u) \in E \\ 0 & \text{else} \end{cases}$$

Hinweis: Initial sollte PR und M' (bzw. M'^T) **stochastisch** und **zeilenweise** (bzw. **spaltenweise**) **stochastisch** normiert sein. Zudem sollten Sie auch (Zwischen-)ergebnisse stochastisch normieren:

$$\text{Stochastische Normalisierung: } \sum_{i=1}^n M_{ij} = 1 \quad \forall \quad j \in 1, 2, \dots, m$$

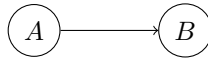
$$\text{Stochastische Normalisierung eines Vektors: } \|v\|_{wsk} = \frac{v_i}{\sum_{j=1}^n v_j} \quad \forall \quad i \in 1, \dots, n$$

$$\text{Stochastische Normalisierung einer Matrix: } \|M'_{ij}\|_{wsk} = \frac{M_{ij}}{\sum_{k=1}^n M_{kj}} \quad \forall \quad i \in 1, \dots, n, \quad j \in 1, \dots, m$$

- 2.3. (6 Punkte) Gegeben Sei der Graph G wie folgt:

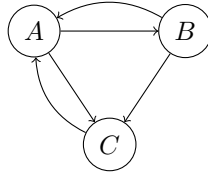
Einführung in die Algorithmik

Übungsblatt 8



Berechnen Sie für die Knoten (Nicht in Matrix Schreibweise) den PageRank nach zwei Iterationen. Nehmen Sie dafür an, dass $\beta = 0.85$ gilt. Geben Sie alle Zwischenschritte und ein normiertes Ergebnis an.

2.4. (6 Punkte) Gegeben Sei der Graph G wie folgt:



Berechnen Sie für alle Knoten (Matrix Schreibweise) den PageRank nach drei Iterationen. Nehmen Sie dafür an, dass $\beta = 0.85$ gilt. Geben Sie alle Zwischenschritte und ein normiertes Ergebnis an.

2.5. (6 Punkte) Implementieren Sie nun den anfangs beschriebenen PageRank Algorithmus. Nutzen Sie dafür nur die in der Einführungsvorlesung zu Python vorgestellten Funktionen.

Mit folgendem Icon können Sie das Programmskelett kopieren. Klicken Sie dafür mit der Linken Maustaste drauf, gegebenenfalls zweimal, oder wählen Sie „Anlage speichern“ nach einem Rechtsklick aus. Verwenden Sie bei Problemen gegebenenfalls ein anderes Programm um die PDF zu öffnen:

```

import networkx as nx
import matplotlib.pyplot as plt
import random

def pagerank(A, d, iterations):
    M = []
    # Calculate M from A

    # Calculate M' = d * M + ...

    # Initialize PR

    # Power Iteration

    return -1

def plot_graph(adj_matrix, pageranks):
    # create directed graph from adjacency matrix
    edges = []
    for i in range(len(adj_matrix)):
        for j in range(len(adj_matrix[i])):
            if adj_matrix[i][j] != 0:
                edges.append((i, j))
    G = nx.DiGraph(edges)
    for i in range(len(pageranks)):
    
```

Einführung in die Algorithmik

Übungsblatt 8

```

        if not G.has_node(i):
            G.add_node(i)
        # set node labels and sizes based on pageranks
        node_sizes, node_labels = [], {}
        for i in G.nodes:
            node_labels[i] = f'{i}\n{pageranks[i]:.2f}'
            node_sizes.append(max(1000, int(10000 * pageranks[i])))
        # set node colors based on pageranks
        node_colors = [(0.0 + pageranks[i], 0.0, 1.0 - pageranks[i], 1.0) for i in G.nodes]
        # plot graph
        pos = nx.spring_layout(G, seed=42, k=1, iterations=10)
        nx.draw_networkx_nodes(G, pos, node_size=node_sizes, node_color=node_colors)
        nx.draw_networkx_labels(G, pos, labels=node_labels, font_size=10)
        nx.draw_networkx_edges(G, pos, connectionstyle='arc3,rad=.1', node_size=node_sizes)
        plt.axis('off')
        plt.show()

def generate_random_digraph(V, E):
    adj_matrix = [[0 for j in range(V)] for i in range(V)]
    edges = set()
    while len(edges) < E:
        start_node, end_node = random.randint(0, V-1), random.randint(0, V-1)
        if start_node != end_node:
            edges.add((start_node, end_node))
    for start_node, end_node in edges:
        adj_matrix[start_node][end_node] = 1
    return adj_matrix

d, iterations = 0.85, 1000
# Wikipedia:
# Die Ergebnisse fuer das englische und deutsche wiki
# sind fuer den gleichen Graphen scheinbar different
# Dies koennte an der (fehlender) Normierung liegen
adj_matrix_wiki = [
    [0,0,0,0,0,0,0,0,0,0,0],#A
    [0,0,1,0,0,0,0,0,0,0,0],#B
    [0,1,0,0,0,0,0,0,0,0,0],#C
    [1,1,0,0,0,0,0,0,0,0,0],#D
    [0,1,0,1,0,1,0,0,0,0,0],#E
    [0,1,0,0,1,0,0,0,0,0,0],#F
    [0,1,0,0,1,0,0,0,0,0,0],#G
    [0,1,0,0,1,0,0,0,0,0,0],#H
    [0,1,0,0,1,0,0,0,0,0,0],#I
    [0,0,0,0,1,0,0,0,0,0,0],#J
    [0,0,0,0,1,0,0,0,0,0,0],#K
]
adj_matrix_stanford = [
    [0,1,1],

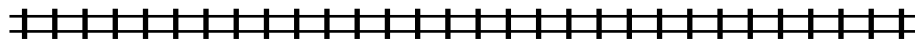
```

Einführung in die Algorithmik

Übungsblatt 8

```
[1,0,1],
[1,0,0],
]
adj_matrix = generate_random_digraph(8,12)
pageranks = pagerank(adj_matrix_wiki, d, iterations)
plot_graph(adj_matrix_wiki, pageranks)
```

3 Zug um Zug zum maximalen Fluss



3.6. (24 Punkte) Züge zählen zu den effizientesten und klimafreundlichsten Fortbewegungs- und Transportmitteln. Daher möchte nun das Logistikunternehmen „Not Just Trains“ auf den Zug aufspringen und die Waren über das Schienennetz transportieren. Dafür hat das Unternehmen in der Datei `train_data.py` alle ihnen zur Verfügung stehenden Verbindungen zwischen Bahnhöfen in großen Städten festgehalten. Für Sie als Programmierende sind besonders die folgenden Schlüssel des Dictionaries relevant:

- **"source"**: Gibt den Startknoten einer Verbindung an.
- **"target"**: Gibt den Endknoten einer Verbindung an.
- **"capacity"**: Gibt die verfügbare Kapazität einer Verbindung an. (Da diese Abhängig von vielen Variablen eines komplexen Systems sind, können sich diese Werte ändern und werden mittels diffiziler Simulationen anhand `def add_capacities()` effizient approximiert.)

Ihre Aufgabe besteht darin die maximal transportierbare Kapazität zwischen zwei Logistikzentren ausfindig zu machen und die für die dazugehörigen Verbindungen notwendigen Bahnhöfe anzugeben.

Haben Sie diese Aufgabe erfolgreich absolviert, müssen Sie noch eine Powerpointpräsentation für das Management erzeugen. Natürlich brauchen Sie dafür Bilder. Diese können Sie sich mit der Funktion `def plot_routes(routes, city_positions, maxflow)` erzeugen lassen. Dabei sind `routes` und `city_positions` die Daten aus `train_data.py` und `maxflow` ein Tupel, welches den Wert des maximalen Flusses und eine Liste mit Bahnhöfen (als Zeichenketten) enthält. Übrigens können die Bilder auch ihnen beim Programmieren und Finden von Fehler helfen.

Die zu implementierenden Funktionen sind:

- (a) `def ford_fulkerson_max_flow(graph, source, sink)`: Die Funktion bekommt einen Graphen (Routen in `train_data.py`), eine Quelle und eine Senke übergeben. Anhand des Ford Fulkerson Algorithmus soll der maximale Fluss zwischen Quelle und Senke berechnet werden, alle dazugehörigen Bahnhöfe (Knoten) ermittelt werden. Geben Sie als Dupel den maximalen Fluss und die Bahnhöfe zurück. (Beta: Geben Sie ein Tripel aus maximalem Fluss, Bahnhöfen und Residualgraph zurück, damit auch der Fluss angezeigt werden kann. Bei der Darstellung in diesem Fall könnte es in Einzelfällen noch zu Fehler kommen).

Erstellen Sie für die Funktion zunächst einen Residualgraphen mit `def create_residual_graph(graph)`. Berechnen Sie anschließend solange es erweiterte Wege gibt diese mit `def find_augmenting_path(graph, source, sink)`. Anschließend sollen Sie den Fluss der erweiterten Wege mit `def calculate_path_flow(graph, path)` berechnen. Aktualisieren Sie danach mit `def update_residual_graph(graph, path, path_flow)` den Residualgraphen.

Einführung in die Algorithmik

Übungsblatt 8

- (b) `def create_residual_graph(graph)`: Die Funktion bekommt einen Graphen übergeben und soll den dazugehörigen initialen Residualgraphen zurückgeben. Der Residualgraph soll ein Dictionary aus Dictionaries sein, welches durch die Iteration über alle Routen des Graphen erstellt wird. Die Idee dabei ist, dass für jede Kante eine Art Rückkante hinzugefügt wird um sowohl die Kapazität als auch den Fluss zu verwalten. **Nehmen Sie an dieser Stelle an, dass es maximal eine Kante zwischen zwei Knoten gibt.** Beispielsweise hat das Ergebnis folgende Struktur:

```
{'Würzburg': {'Bamberg': 84, 'Erlangen': 82},
 'Bamberg': {'Würzburg': 0, 'Erlangen': 38},
 'Erlangen': {'Würzburg': 0, 'Bamberg': 0, 'Nürnberg': 64},
 ...
}
```

- (c) `def find_augmenting_path(graph, source, sink)`: Diese Funktion bekommt einen (Residual-)Graphen, die Quelle und die Senke übergeben und soll einen Pfad zurückgeben. Mittels Tiefensuche soll die Funktion einen augmentierenden Pfad finden. Das ist ein Pfad anhand dessen der bestehende Fluss verbessert werden kann, also ein vergrößernder oder flusserhöhender Pfad. Beispielsweise kann der Pfad wie folgt aussehen:
- ```
['Würzburg', 'Erlangen', 'Nürnberg', 'Ingolstadt', 'Regensburg', 'Passau']
```
- (d) `def calculate_path_flow(graph, path)`: Die Funktion bekommt einen (Residual-)Graphen und einen Pfad und gibt den Fluss über den Pfad zurück. Dazu wird über alle Verbindungen im Pfad iteriert um die kleinste Kapazität zu ermitteln.
- (e) `def update_residual_graph(graph, path, path_flow)`: Diese Funktion bekommt einen (Residual-)Graphen, einen Pfad und einen Flusswert übergeben und aktualisiert die Flusswerte im übergebenen Graphen. Dazu sollen die Kapazitäten der Verbindungen im übergebenen Pfad mit dem Flusswert aktualisiert werden. Sie müssen an dieser Stelle nichts zurückgeben, da Python bei Dictionaries referenzbasiert die Objekte übergibt (Call by reference).
- (f) `def transform_graph(graph)`: Bearbeiten Sie diese Aufgaben erst nachdem alle anderen Funktionen funktionieren. Der Algorithmus kann durch eine kleine Veränderung auch mit antiparallelen Kanten (siehe Begleitliteratur) umgehen. Ergänzen Sie dafür in `def ford_fulkerson_max_flow(graph, source, sink)` anfangs vor allen anderen Operationen die Zeile `graph = transform_graph(graph.copy())`, welche eine Kopie des Graphen erstellt. Implementieren Sie anschließend `def transform_graph(graph)` so, dass Sie mit einer verschachtelten Iteration über die Kanten existierende antiparallele Kanten erkennen und eine davon durch einen Hilfsknoten und zwei Kanten ersetzen. Es genügt, wenn die hinzugefügten Verbindungen nur die drei Schlüssel `"source"`, `"target"`, `"capacity"` haben. Benennen Sie den Hilfsknoten so, dass er bei einer Verbindung von  $u$  nach  $v$  den Namen `u["source"] + u["target"] + "helper"` erhält.

Mit folgendem Icon können Sie das Programmskelett kopieren. Klicken Sie dafür mit der Linken Maustaste drauf, gegebenenfalls zweimal, oder wählen Sie „Anlage speichern“ nach einem Rechtsklick aus. Verwenden Sie bei Problemen gegebenenfalls ein anderes Programm um die PDF zu öffnen: 

Zudem sind die Datensätze der Verbindungen  und die unternehmensinternen Funktionen  gegeben. Die Daten sehen beispielsweise wie folgt aus:

```
Routes_Bavaria = [
 {"source": "Würzburg", "target": "Bamberg", "color": "blue", "size": 1},
 {"source": "Würzburg", "target": "Erlangen", "color": "red", "size": 2},
 {"source": "Bamberg", "target": "Erlangen", "color": "orange", "size": 1},
 {"source": "Erlangen", "target": "Nürnberg", "color": "purple", "size": 2},
 {"source": "Nürnberg", "target": "Augsburg", "color": "yellow", "size": 3},
```

# Einführung in die Algorithmik

## Übungsblatt 8

```
{"source": "Nürnberg", "target": "Ingolstadt", "color": "cyan", "size": 4},
...
```

---

```
from train_max_flow_backend import *
```

```
def ford_fulkerson_max_flow(graph, source, sink):
 #graph = transform_graph(graph.copy())
 pass
```

```
def create_residual_graph(graph):
 pass
```

```
def find_augmenting_path(graph, source, sink):
 pass
```

```
def calculate_path_flow(graph, path):
 pass
```

```
def update_residual_graph(graph, path, path_flow):
 pass
```

```
def transform_graph(graph):
 pass
```

```

Testbereich beginnt:
#####
```

```
#from train_data import Routes_Bavaria as Routes, city_positions_bavaria as city_positions
```

```
#from train_data import Routes_Bavaria_Extended as Routes
#from train_data import city_positions_bavaria_extended as city_positions
```

```
#from train_data import Routes, city_positions
```

```
#add_capacities(Routes)
```

```
#maxflow = ford_fulkerson_max_flow(Routes, "Würzburg", "Passau")
#maxflow = ford_fulkerson_max_flow(Routes, "Vancouver", "Miami")
```

```
from train_data import Simple_Test_Case as Routes
from train_data import simple_test_case_positions as city_positions
```

```
maxflow= ford_fulkerson_max_flow(Routes, "A", "D")
```

```
plot_routes(Routes, city_positions, maxflow)
```

---